

Homework Assignment 1

Random Walks and the Central Limit Theorem

EC327 Summer 2026

Introduction

Assignment goals

By the end of this assignment you will

- have written, compiled, and run a real C++ program on your Multipass/WSL2 development environment
- understand the anatomy of a C++ source file: `#include`, `int main()`, declarations vs. statements, return values
- have defined your own `struct` and passed it by reference into functions
- have used the `<random>` library to generate random numbers in a reproducible way
- have used `std::vector` and the range-based `for` loop
- have built a multi-file project with `CMake`
- have **seen the Central Limit Theorem appear in your own data** — the histogram of final positions for many random walkers takes on the shape of a Gaussian (bell curve)

The conceptual goal — the CLT thing — is the most important one. Every statistical engineering tool you will ever use is built on the CLT. In EK381 you will prove it. Here you will see it in code that you wrote.

Working style

Individual assignment. Talk with classmates about ideas and approaches; write your own code.

AI use is expected and encouraged, per the course's AI-in-the-learning-loop policy. You must disclose your AI use and you must be able to explain every line of your submission. See the AI disclosure section below.

Due date

Thursday, May 28, 2026, 11:59 PM (end of L6).

Assignment value

Graded on a 100-point internal scale. HW1 contributes **6.25 course points** (one of four HWs in the 25% HW component).

Late policy

30% flat penalty for any submission up to 1 week late. No credit after that. Documented legitimate excuses (illness, family emergency, etc.) waive the penalty — email the instructor.

Submission

Submit via the course web app at curl.bu.edu/ec327 (link pending — instructor will share before HW1 is due). Submit all required source files plus your reflection and AI disclosure.

How this assignment is graded — and the bigger picture

HW1 is the **most heavily scaffolded assignment of the term**. The parts are spelled out, function signatures are given, the common pitfalls are named in advance, and a `smoke_test.py` ships with your starter code so you can self-check before submitting. By HW4 you will receive a much terser spec and will write your own correctness tests. The arc is deliberate and described in the syllabus under “**Scaffolding tapers across the term.**”

The framing: every HW is a **product ship**. The class is your engineering team; the instructor and AI together play QA, customer, and reviewer. Different customers provide different levels of detail. HW1 is the *internal-customer-with-tight-contract* end of that spectrum — a full spec plus acceptance tests. The week-6 project is the *startup-ship-it* end — a wish and a deadline. The variation across the term is part of the curriculum, not an inconsistency. Real engineers deliver in both modes and learn to recognize which they’re in.

Background — what you are building

A **random walker** is a thing that takes a step in a random direction at each tick of a clock. Imagine someone standing on a number line at position 0; at each tick they flip a fair coin; heads they move +1, tails they move −1. After N steps, where are they?

The interesting question is not where any *one* walker ends up — the interesting question is the *distribution* of where many independent walkers end up. If you run K walkers each for N steps and plot a histogram of their final positions, what shape do you get?

You will discover, by running your own code, that the histogram looks like a **bell curve** — a Gaussian — regardless of what N you pick. This is the **Central Limit Theorem** at work: the sum of many independent, identically distributed random variables approaches a Gaussian, no matter what the underlying distribution is.

You will also observe that the *spread* of the histogram grows like $\sqrt{N}$. Doubling the number of steps grows the standard deviation by $\sqrt{2}$, not by 2. This is one of the most important rules of thumb in measurement science.

The point of this homework is to build the simulation, watch these properties emerge from the data you generated, and have the right reaction: *“oh, so that’s where the Gaussian comes from.”*

Setup

You should have completed **Lab 0** (Multipass / WSL2 standup) before starting HW1. If you have not, do that first — Lab 0 is ungraded and gives you the development environment you need.

You will receive a starter directory containing:

```
hw1_random_walk/
├─ CMakeLists.txt      # builds the whole thing – you may extend it
├─ Walker.h            # struct + function declarations, with TODOs
├─ Walker.cpp          # function implementations, with TODOs
├─ Histogram.h         # added in Part 4
├─ Histogram.cpp       # added in Part 4
└─ hw1_main.cpp        # orchestrates the experiment
```

Build with:

```
cmake -B build
cmake --build build
./build/hw1_main
```

If you see a Gaussian-looking histogram printed to your terminal, you are done. If you don’t, keep working.

Part 1 — One walker, one step at a time (20 points)

Goal: the simplest possible thing that works. One walker, ten steps, printed.

Define the `Walker` struct in `Walker.h`:

```
struct Walker {
    int position;
};
```

Implement a `step` function in `Walker.cpp` that takes a `Walker` by reference and uses the `<random>` library to add `+1` or `-1` to its position with equal probability:

```
void step(Walker& w);
```

In `hw1_main.cpp`, create one `Walker` at position 0, call `step` on it 10 times, and print the position after each step.

Expected output (your numbers will differ — that is the point):

```
Walker at position: 0
After step 1: 1
After step 2: 0
After step 3: -1
After step 4: 0
After step 5: 1
After step 6: 0
After step 7: -1
After step 8: -2
After step 9: -3
After step 10: -2
```

New C++ vocabulary in this part: struct definition · pass-by-reference (&) · <random> (std::mt19937, std::uniform_int_distribution) · function declaration in header / definition in .cpp

Hint on the random-number generator: declare the engine *outside* of `step` (e.g., as a static local, or in your `main` and pass it in). If you re-create the engine on every call, every call produces the same number — a classic first-time-with-<random> pitfall.

Part 2 — One walker, N steps (15 points)

Goal: parameterize the number of steps. Vary `N`. Notice the variability.

Add a function in `Walker.cpp`:

```
void walk_for(Walker& w, int N);
```

that calls `step` `N` times.

In `hw1_main.cpp`, replace the 10-step demo with a loop that runs three walkers:

- one for `N = 10` steps
- one for `N = 100` steps
- one for `N = 1000` steps

Print the final position of each. **Run your program three times.** Note that you get a different answer each time. Why?

Expected output (one run):

```
N=10:    final position 2
N=100:   final position -14
N=1000:  final position 38
```

New C++ vocabulary: function with multiple parameters · `const int` for unchanging values · the intuition that “one trial tells you nothing”

Part 3 — K independent walkers (25 points)

Goal: repeat the experiment K times and compute summary statistics.

Add a function in `Walker.cpp`:

```
std::vector<int> run_trials(int K, int N);
```

that runs K independent walkers, each for N steps starting at position 0, and returns a vector of K final positions.

In `hw1_main.cpp`, compute the **mean** and the **sample variance** of the final positions using hand-written for loops (don't reach for `<algorithm>` yet — that's the topic of the Lecture 7 handout, lifted in HW2; the point here is to write the accumulator by hand).

Recall:

$$\text{mean} = \frac{1}{K} \sum_{i=1}^K x_i \quad \text{variance} = \frac{1}{K-1} \sum_{i=1}^K (x_i - \text{mean})^2$$

Run with K = 10, K = 100, K = 1000, all with N = 100 steps.

Expected output (numbers will vary):

```
K=10,   N=100:  mean = -1.4    variance = 84.7
K=100,  N=100:  mean = 0.32    variance = 98.1
K=1000, N=100:  mean = 0.05    variance = 100.3
```

What do you notice as K grows? (Hint: with N = 100, the *true* variance is exactly 100. Your estimate gets closer.)

New C++ vocabulary: `std::vector<int>` · range-based for (`int x : v`) · the accumulator pattern · sample-variance formula

Part 4 — Terminal histogram + the CLT moment (30 points)

Goal: see the Central Limit Theorem with your own eyes.

Create two new files, `Histogram.h` and `Histogram.cpp`, containing:

```
std::map<int, int> histogram(const std::vector<int>& positions, int bucket_width);
void print_histogram(const std::map<int, int>& h);
```

`histogram` bins the positions into buckets of the given width (e.g., `bucket_width = 5` means positions -2, 0, 3 all go into bucket 0; positions 5, 7, 9 all go into bucket 5; positions -10, -7 all go into bucket -10).

`print_histogram` prints a sideways bar chart with * characters, one row per bucket, sorted by bucket.

In `hw1_main.cpp`, run the experiment with **K = 10000 walkers**, **N = 100 steps**, **bucket_width = 5**, and print the histogram. Include the mean and variance.

Expected output (rough shape — your numbers will differ):

K=10000, N=100, bucket=5

```
bucket  count
-35:    *
-30:   ****
-25:  *****
-20:  *****
-15:  *****
-10:  *****
-5:   *****
0:    *****
5:    *****
10:   *****
15:   *****
20:   *****
25:   *****
30:   ****
35:    *
```

mean = 0.04, variance = 99.8

In your output.txt (see Required submission below), capture the histogram for at least three different (K, N) combinations — for example (K=10000, N=100), (K=10000, N=400), (K=10000, N=1600). Notice how the histogram **widens by a factor of 2** when N **quadruples** (the $\sqrt{N}$ scaling).

Write a 1–2 sentence observation in your output file: what shape is the histogram? Why?

New C++ vocabulary: `const` reference parameter (`const std::vector<int>&`) · `std::map<int, int>` for sparse indexed counts · output formatting with `std::cout` · integer bucketing math

On `std::map` vs `std::vector`: you can use either. `std::map<int, int>` is the cleaner choice because the bucket index can be negative and unbounded; `std::vector<int>` works if you offset the index. Use whichever you find more readable.

Part 5 — Multi-file build + reflection (10 points)

Goal: put the project into the shape of a real C++ project, and articulate what you observed.

By this point your code is already in `Walker.h` / `Walker.cpp` / `Histogram.h` / `Histogram.cpp` / `hw1_main.cpp`. Make sure:

- every `.h` file has `#pragma once` (or include guards) at the top
- every `.h` file declares functions, never defines them
- every `.cpp` file `#includes` its own header and implements the declared functions

- `hw1_main.cpp` #includes both headers
- the provided `CMakeLists.txt` builds everything together — extend it if you added files

Write reflection.md — exactly 100 words (within ± 20) on what you observed. Required to address:

- What did the histogram look like? Did the shape match your expectation?
- How did the variance grow with N ? Did the standard deviation scale as $\sqrt{N}$?
- One thing about C++ that surprised you, frustrated you, or that you want to know more about.

This is the conceptual deliverable. The point of the entire homework lives here.

Required submission

Submit a **single .zip file**, uploaded at curl.bu.edu/ec327, containing:

File	What it is
<code>Walker.h Walker.cpp</code>	The walker struct + step / walk_for / run_trials
<code>Histogram.h Histogram.cpp</code>	The histogram functions
<code>hw1_main.cpp</code>	The main experiment driver
<code>CMakeLists.txt</code>	Build configuration (may be the unmodified starter)
<code>output.txt</code>	Captured terminal output for at least 3 (K, N) combinations
<code>reflection.md</code>	Your 100-word reflection
<code>ai_disclosure.md</code>	Your AI use disclosure (template below)

Do **not** submit the `build/` directory or any compiled binaries. Upload **one .zip** — loose files and `.tar.gz` are rejected by the uploader.

A helper, `ec327_zip.py`, ships in your starter package. From your project directory:

```
python3 ec327_zip.py build hw1      # writes hw1_submission.zip + checks it
python3 ec327_zip.py check hw1     # re-check an existing zip
```

It bundles the required source files (and picks up `output.txt`, `reflection.md`, `ai_disclosure.md` if present), skips the `build/` directory, and prints a present/missing checklist. You can also zip by hand — the upload page only requires a single valid `.zip`, and shows the same checklist against the files *inside* it after you upload.

Testing your work — the smoke test

Your starter package includes `smoke_test.py`. Run it before submitting:

python3 smoke_test.py

It does the following — nothing more, nothing less:

- Checks that the required source files exist
- Runs `cmake -B build && cmake --build build` and verifies the build succeeds
- Runs your binary and captures its output
- Looks for `mean = X` and `variance = Y` lines and verifies the numbers are statistically plausible (mean near zero, variance in a sane range)
- Verifies the output contains a histogram (lines containing `*`)

Passing the smoke test is necessary, not sufficient. The instructor will read your code, check for clarity, check the bucketing edge cases in Part 4, read your reflection, and run additional (`K`, `N`) combinations during grading. A submission that passes the smoke test can still earn a low score if the code is unreadable, the bucketing is wrong on edge cases, or the reflection is one sentence long.

The smoke test is your **first line of defense** — the way CI is for a real engineering team. Run it. Pass it. Then submit. “It worked on my machine but the smoke test failed” is not a defense; the smoke test runs in the same environment we will grade in.

What the smoke test deliberately does **not** check: - Code style (we run `clang-format` separately) - Correctness of negative-index bucketing (the part-4 gotcha) - Whether your reflection actually addresses the prompts - Whether your AI disclosure is honest

Those are instructor-grading territory.

Grading rubric

Part	Pts	Earns full credit	Common deductions
1	20	<code>Walker</code> struct defined; <code>step</code> correctly uses <code><random></code> with the engine declared outside the function; main prints 10 steps with reasonable-looking output	RNG re-seeded every call (gives same number every step) · <code>step</code> doesn't take a reference (so the walker's position never changes back in main) · forgets <code>#include <random></code>
2	15	<code>walk_for(Walker&, int)</code> works; three runs at <code>N = 10, 100, 1000</code> produce visibly different final positions	off-by-one in the step count · uses <code>walk_for(w, 10)</code> then prints <code>w.position</code> without resetting <code>w</code> for the next run

Part	Pts	Earns full credit	Common deductions
3	25	<code>run_trials(int, int)</code> returns a vector of the right size; mean and variance hand-computed correctly; tested with $K = 10, 100, 1000$	uses K in the variance denominator instead of $K-1$ (the sample-variance gotcha — partial credit if recognized in the reflection) · doesn't reset walker position between trials · integer division when computing mean
4	30	Histogram correctly bins (especially handles negative positions and 0 correctly); print is readable; runs at $K = 10000$ with at least 3 N values; observation sentence is present	bucket-width math wrong (e.g., positions -1 and 1 both end up in bucket 0 when they should be in bucket 0 and bucket 0 — yes, that one is fine — but -3 and 3 should be in different buckets with width 5: think it through) · histogram is unreadable · no observation sentence
5	10	Project is properly split; headers vs. implementation separation is clean; <code>reflection.md</code> is between 80 and 120 words and addresses all three required points; AI disclosure complete	everything dumped in <code>hw1_main.cpp</code> · headers contain function definitions (multiple-definition link errors) · reflection is one sentence long · no AI disclosure

Style baseline (across all parts): your code must compile with `-Wall -Wextra` without warnings. Apply `clang-format` with the course `.clang-format` file (provided in the starter). Don't worry about `clang-tidy` yet — that's Lab 2 territory.

Hints and pitfalls

The RNG must be persistent across calls. The single most common HW1 bug. Wrong:

```
void step(Walker& w) {
    std::mt19937 rng(0); // re-seeded every call!
    std::uniform_int_distribution<int> dist(0, 1);
    w.position += (dist(rng) == 0) ? -1 : 1;
}
```

Right (one approach — there are others):

```
void step(Walker& w) {
    static std::mt19937 rng(std::random_device{}());
    static std::uniform_int_distribution<int> dist(0, 1);
    w.position += (dist(rng) == 0) ? -1 : 1;
}
```

The `static` keyword inside a function says “create this once, on the first call, and reuse the same one for every subsequent call.”

Mean and sample variance need careful loops:

```
double mean = 0.0;
for (int x : positions) mean += x;
mean /= positions.size();

double var = 0.0;
for (int x : positions) var += (x - mean) * (x - mean);
var /= (positions.size() - 1); // note K-1, not K
```

Why K-1? Because you used the data to estimate the mean, so you have one less “degree of freedom” left for estimating the variance. EK381 will derive this; for now, just use it.

Bucketing with negative positions is the part-4 gotcha. With `bucket_width = 5`, position `-3` should go into bucket `-5` (i.e., the bucket that *contains* positions from `-5` to `-1` inclusive). Position `0` should go into bucket `0`. Position `4` should go into bucket `0`. Position `5` should go into bucket `5`. **Integer division in C++ rounds toward zero, not toward negative infinity** — so `(-3) / 5` gives `0` in C++, but you want `-1`. Be careful.

One way to handle this cleanly:

```
int bucket_of(int x, int width) {
    if (x >= 0) return (x / width) * width;
    return -(((x - 1) / width) + 1) * width;
}
```

(Test this. There may be a cleaner formulation.)

Library check — the Lecture 7 handout (and HW2) show that

```
double mean = std::accumulate(positions.begin(), positions.end(), 0.0) / positions.size();
```

is the one-line standard-library version of your accumulator loop. For HW1, **do not** use `std::accumulate` — write the loop by hand so you understand what is happening. We will

come back to this.

AI disclosure (required)

Submit `ai_disclosure.md` with the following template filled out. ~100 words total.

`## AI use disclosure for HW1`

`### Roles used`

(check all that apply)

- `[ ]` Pair programmer – AI drafted or completed code with me
- `[ ]` Coach – AI explained concepts to me
- `[ ]` IT aide – AI helped me with environment / compiler / build issues
- `[ ]` Code reviewer – AI critiqued code I had written
- `[ ]` Test-case generator – AI suggested edge cases
- `[ ]` Spec clarifier – AI helped me unpack ambiguities in the assignment
- `[ ]` Rubber duck – I explained the problem to AI and figured it out mid-explanation
- `[ ]` Cross-language translator – AI showed me C++ equivalents of Python idioms

`### What I accepted, what I rejected`

(2–3 sentences. Where did AI help? Where did it suggest something wrong or unhelpful, and how did you catch it?)

`### Confidence`

(1 sentence. Can you walk through every line of your submission and explain what it does and why?)

The disclosure is the practice of the AI policy, not a hurdle. There is no penalty for *using* AI heavily; there is a penalty for using it without disclosing, or for submitting code you cannot explain. See the course AI policy for the full statement.

What comes next

HW1's `Walker` struct is the seed of HW2. In HW2 you will extend it to 2D (`int x, y`), let it take steps in four directions, and watch the cloud of walkers spread out as $\sqrt{t}$ — Brownian motion. Don't throw your HW1 code away.